

Suppose that L is a subset of I^* , where I is a nonempty set of symbols. If $x \in I^*$, we let $L/x = \{z \in I^* \mid xz \in L\}$. We say that the strings $x \in I^*$ and $y \in I^*$ are **distinguishable with respect to L** if $L/x \neq L/y$. A string z for which $xz \in L$ but $yz \notin L$, or $xz \notin L$, but $yz \in L$ is said to **distinguish** x and y with respect to L . When $L/x = L/y$, we say that x and y are **indistinguishable** with respect to L .

27. Let L be the set of all bit strings that end with 01. Show that 11 and 10 are distinguishable with respect to L and that the strings 1 and 11 are indistinguishable with respect to L .
28. Suppose that $M = (S, I, f, s_0, F)$ is a deterministic finite-state machine. Show that if x and y are two strings in I^* that are distinguishable with respect to $L(M)$, then $f(s_0, x) \neq f(s_0, y)$.
- *29. Suppose that L is a subset of I^* and for some positive integer n there are n strings in I^* such that every two of these strings are distinguishable with respect to L . Prove that every deterministic finite-state automaton recognizing L has at least n states.
- *30. Let L_n be the set of strings with at least n bits in which the n th symbol from the end is a 0. Use Exercise 29 to show that a deterministic finite-state machine recognizing L_n must have at least 2^n states.
- *31. Use Exercise 29 to show that the language consisting of all bit strings that are palindromes (that is, strings that equal their own reversals) is not regular.

13.5 Turing Machines

13.5.1 Introduction



The finite-state automata studied earlier in this chapter cannot be used as general models of computation. They are limited in what they can do. For example, finite-state automata are able to recognize regular sets, but are not able to recognize many easy-to-describe sets, including $\{0^n 1^n \mid n \geq 0\}$, which computers recognize using memory. We can use finite-state automata to compute relatively simple functions such as the sum of two numbers, but we cannot use them to compute functions that computers can, such as the product of two numbers. To overcome these deficiencies we can use a more powerful type of machine known as a Turing machine, after Alan Turing, the illustrious mathematician and computer scientist who invented them in the 1930s.

Basically, a Turing machine consists of a control unit, which at any step is in one of finitely many different states, together with a tape divided into cells, which is infinite in both directions. Turing machines have read and write capabilities on the tape as the control unit moves back and forth along this tape, changing states depending on the tape symbol read. Turing machines are more powerful than finite-state machines because they include memory capabilities that finite-state machines lack. We will show how to use Turing machines to recognize sets, including sets that cannot be recognized by finite-state machines. We will also show how to compute functions using Turing machines. Turing machines are the most general models of computation; essentially, they can do whatever a computer can do. Note that Turing machines are much more powerful than real computers, which have finite memory capabilities.

“Machines take me by surprise with great frequency.” – Alan Turing

13.5.2 Definition of Turing Machines

We now give the formal definition of a Turing machine. Afterward we will explain how this formal definition can be interpreted in terms of a control head that can read and write symbols on a tape and move either right or left along the tape.

Definition 1

A Turing machine $T = (S, I, f, s_0)$ consists of a finite set S of states, an alphabet I containing the blank symbol B , a partial function f from $S \times I$ to $S \times I \times \{R, L\}$, and a starting state s_0 .

Recall from Section 2.3 that a partial function is defined only for those elements in its domain of definition. This means that for some (state, symbol) pairs the partial function f may be

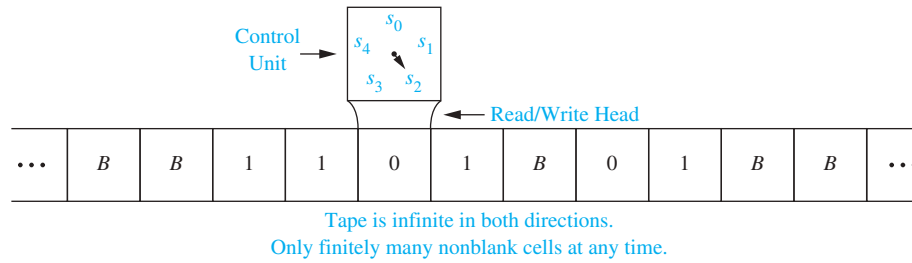


FIGURE 1 A representation of a Turing machine.

undefined, but for a pair for which it is defined, there is a unique (state, symbol, direction) triple associated to this pair. We call the five-tuples corresponding to the partial function in the definition of a Turing machine the **transition rules** of the machine.

To interpret this definition in terms of a machine, consider a control unit and a tape divided into cells, infinite in both directions, having only a finite number of nonblank symbols on it at any given time, as pictured in Figure 1. The action of the Turing machine at each step of its operation depends on the value of the partial function f for the current state and tape symbol.

At each step, the control unit reads the current tape symbol x . If the control unit is in state s and if the partial function f is defined for the pair (s, x) with $f(s, x) = (s', x', d)$, the control unit

1. enters the state s' ,
2. writes the symbol x' in the current cell, erasing x , and
3. moves right one cell if $d = R$ or moves left one cell if $d = L$.

We write this step as the five-tuple (s, x, s', x', d) . If the partial function f is undefined for the pair (s, x) , then the Turing machine T will *halt*.

A common way to define a Turing machine is to specify a set of five-tuples of the form (s, x, s', x', d) . The set of states and input alphabet is implicitly defined when such a definition is used.

At the beginning of its operation a Turing machine is assumed to be in the initial state s_0 and to be positioned over the leftmost nonblank symbol on the tape. If the tape is all blank, the control head can be positioned over any cell. We will call the positioning of the control head over the leftmost nonblank tape symbol the *initial position* of the machine.

Example 1 illustrates how a Turing machine works.

EXAMPLE 1 What is the final tape when the Turing machine T defined by the seven five-tuples $(s_0, 0, s_0, 0, R)$, $(s_0, 1, s_1, 1, R)$, (s_0, B, s_3, B, R) , $(s_1, 0, s_0, 0, R)$, $(s_1, 1, s_2, 0, L)$, (s_1, B, s_3, B, R) , and $(s_2, 1, s_3, 0, R)$ is run on the tape shown in Figure 2(a)?

Extra Examples ➤

Solution: We start the operation with T in state s_0 and with T positioned over the leftmost nonblank symbol on the tape. The first step, using the five-tuple $(s_0, 0, s_0, 0, R)$, reads the 0 in the leftmost nonblank cell, stays in state s_0 , writes a 0 in this cell, and moves one cell right. The second step, using the five-tuple $(s_0, 1, s_1, 1, R)$, reads the 1 in the current cell, enters state s_1 , writes a 1 in this cell, and moves one cell right. The third step, using the five-tuple $(s_1, 0, s_0, 0, R)$, reads the 0 in the current cell, enters state s_0 , writes a 0 in this cell, and moves one cell right. The fourth step, using the five-tuple $(s_0, 1, s_1, 1, R)$, reads the 1 in the current cell, enters state s_1 , writes a 1 in this cell, and moves right one cell. The fifth step, using the five-tuple $(s_1, 1, s_2, 0, L)$, reads the 1 in the current cell, enters state s_2 , writes a 0 in this cell, and moves left one cell. The sixth step, using the five-tuple $(s_2, 1, s_3, 0, R)$, reads the 1 in the current cell, enters the state s_3 , writes a 0 in this cell, and moves right one cell. Finally, in the seventh step, the machine halts because there is no five-tuple beginning with the pair $(s_3, 0)$ in the description of the machine. The steps are shown in Figure 2.

Note that T changes the first pair of consecutive 1s on the tape to 0s and then halts. ◀

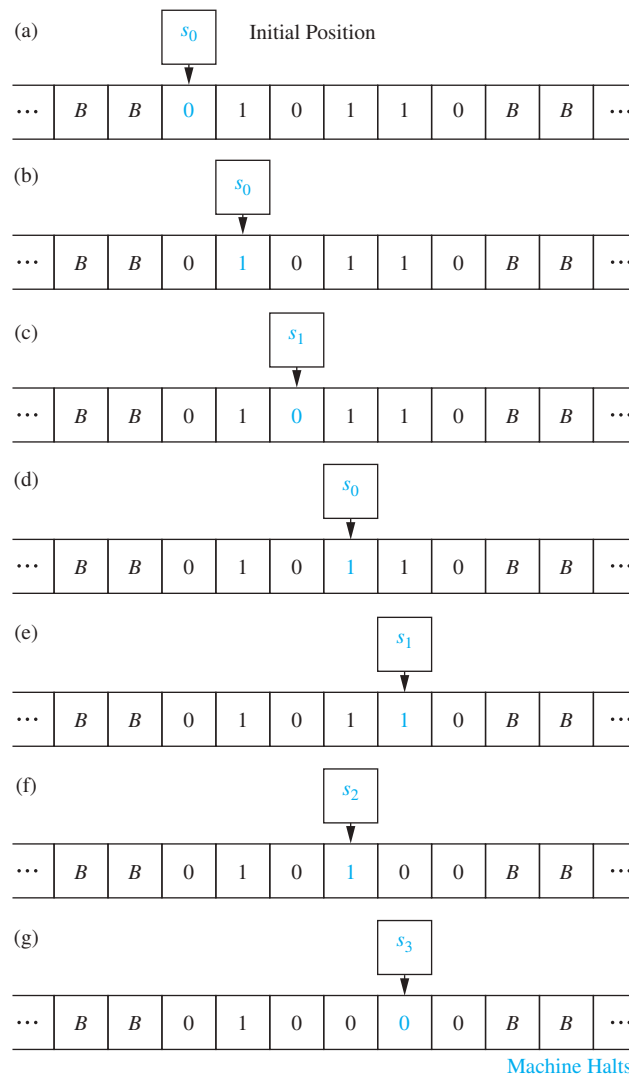


FIGURE 2 The steps produced by running T on the tape in Figure 1.

13.5.3 Using Turing Machines to Recognize Sets

Turing machines can be used to recognize sets. To do so requires that we define the concept of a final state as follows. A *final state* of a Turing machine T is a state that is not the first state in any five-tuple in the description of T using five-tuples (for example, state s_3 in Example 1).

We can now define what it means for a Turing machine to recognize a string. Given a string, we write consecutive symbols in this string in consecutive cells.

Definition 2

Let V be a subset of an alphabet I . A Turing machine $T = (S, I, f, s_0)$ recognizes a string x in V^* if and only if T , starting in the initial position when x is written on the tape, halts in a final state. T is said to recognize a subset A of V^* if x is recognized by T if and only if x belongs to A .

Note that to recognize a subset A of V^* we can use symbols not in V . This means that the input alphabet I may include symbols not in V . These extra symbols are often used as markers (see Example 3).

When does a Turing machine T *not* recognize a string x in V^* ? The answer is that x is not recognized if T does not halt or halts in a state that is not final when it operates on a tape containing the symbols of x in consecutive cells, starting in the initial position. (The reader should understand that this is one of many possible ways to define how to recognize sets using Turing machines.)

We illustrate this concept with Example 2.

EXAMPLE 2 Find a Turing machine that recognizes the set of bit strings that have a 1 as their second bit, that is, the regular set $(0 \cup 1)1(0 \cup 1)^*$.

Solution: We want a Turing machine that, starting at the leftmost nonblank tape cell, moves right, and determines whether the second symbol is a 1. If the second symbol is 1, the machine should move into a final state. If the second symbol is not a 1, the machine should not halt or it should halt in a nonfinal state.

To construct such a machine, we include the five-tuples $(s_0, 0, s_1, 0, R)$ and $(s_0, 1, s_1, 1, R)$ to read in the first symbol and put the Turing machine in state s_1 . Next, we include the five-tuples $(s_1, 0, s_2, 0, R)$ and $(s_1, 1, s_3, 1, R)$ to read in the second symbol and either move to state s_2 if this symbol is a 0, or to state s_3 if this symbol is a 1. We do not want to recognize strings that have a 0 as their second bit, so s_2 should not be a final state. We want s_3 to be a final state. So, we can include the five-tuple $(s_2, 0, s_2, 0, R)$. Because we do not want to recognize the empty string or a string with one bit, we also include the five-tuples $(s_0, B, s_2, 0, R)$ and $(s_1, B, s_2, 0, R)$.

The Turing machine T consisting of the seven five-tuples listed here will terminate in the final state s_3 if and only if the bit string has at least two bits and the second bit of the input string is a 1. If the bit string contains fewer than two bits or if the second bit is not a 1, the machine will terminate in the nonfinal state s_2 . 

Given a regular set, a Turing machine that always moves to the right can be built to recognize this set (as in Example 2). To build the Turing machine, first find a finite-state automaton that recognizes the set and then construct a Turing machine using the transition function of the finite-state machine, always moving to the right.

We will now show how to build a Turing machine that recognizes a nonregular set.

EXAMPLE 3 Find a Turing machine that recognizes the set $\{0^n 1^n \mid n \geq 1\}$.



Solution: To build such a machine, we will use an auxiliary tape symbol M as a marker. We have $V = \{0, 1\}$ and $I = \{0, 1, M\}$. We wish to recognize only a subset of strings in V^* . We will have one final state, s_6 . The Turing machine successively replaces a 0 at the leftmost position of the string with an M and a 1 at the rightmost position of the string with an M , sweeping back and forth, terminating in a final state if and only if the string consists of a block of 0s followed by a block of the same number of 1s.

Although this is easy to describe and is easily carried out by a Turing machine, the machine we need to use is somewhat complicated. We use the marker M to keep track of the leftmost and rightmost symbols we have already examined. The five-tuples we use are $(s_0, 0, s_1, M, R)$, $(s_1, 0, s_1, 0, R)$, $(s_1, 1, s_1, 1, R)$, (s_1, M, s_2, M, L) , (s_1, B, s_2, B, L) , $(s_2, 1, s_3, M, L)$, $(s_3, 1, s_3, 1, L)$, $(s_3, 0, s_4, 0, L)$, (s_3, M, s_5, M, R) , $(s_4, 0, s_4, 0, L)$, (s_4, M, s_0, M, R) , and (s_5, M, s_6, M, R) . For example, the string 000111 would successively become $M00111$, $M0011M$, $MM011M$, $MM01MM$, $MMM1MM$, $MMMMMM$ as the machine operates until it halts. Only the changes are shown, as most steps leave the string unaltered.

We leave it to the reader (Exercise 13) to explain the actions of this Turing machine and to explain why it recognizes the set $\{0^n 1^n \mid n \geq 1\}$. 

It can be shown that a set can be recognized by a Turing machine if and only if it can be generated by a type 0 grammar, or in other words, if the set is generated by a phrase-structure grammar. The proof will not be presented here.

13.5.4 Computing Functions with Turing Machines

A Turing machine can be thought of as a computer that finds the values of a partial function. To see this, suppose that the Turing machine T , when given the string x as input, halts with the string y on its tape. We can then define $T(x) = y$. The domain of T is the set of strings for which T halts; $T(x)$ is undefined if T does not halt when given x as input. Thinking of a Turing machine as a machine that computes the values of a function on strings is useful, but how can we use Turing machines to compute functions defined on integers, on pairs of integers, on triples of integers, and so on?

To consider a Turing machine as a computer of functions from the set of k -tuples of nonnegative integers to the set of nonnegative integers (such functions are called **number-theoretic functions**), we need a way to represent k -tuples of integers on a tape. To do so, we use **unary representations** of integers. We represent the nonnegative integer n by a string of $n + 1$ 1s so that, for instance, 0 is represented by the string 1 and 5 is represented by the string 11111. To represent the k -tuple $(n_1, n_2, \dots, n_k)$, we use a string of $n_1 + 1$ 1s, followed by an asterisk, followed by a string of $n_2 + 1$ 1s, followed by an asterisk, and so on, ending with a string of $n_k + 1$ 1s. For example, to represent the four-tuple $(2, 0, 1, 3)$ we use the string 111 * 1 * 11 * 1111.

We can now consider a Turing machine T as computing a sequence of number-theoretic functions $T, T^2, \dots, T^k, \dots$. The function T^k is defined by the action of T on k -tuples of integers represented by unary representations of integers separated by asterisks.

EXAMPLE 4 Construct a Turing machine for adding two nonnegative integers.

Solution: We need to build a Turing machine T that computes the function $f(n_1, n_2) = n_1 + n_2$. The pair (n_1, n_2) is represented by a string of $n_1 + 1$ 1s followed by an asterisk followed by $n_2 + 1$ 1s. The machine T should take this as input and produce as output a tape with $n_1 + n_2 + 1$ 1s. One way to do this is as follows. The machine starts at the leftmost 1 of the input string, and carries out steps to erase this 1, halting if $n_1 = 0$ so that there are no more 1s before the asterisk, replaces the asterisk with the leftmost remaining 1, and then halts. We can use these five-tuples to do this: $(s_0, 1, s_1, B, R)$, $(s_1, *, s_2, B, R)$, $(s_1, 1, s_2, B, R)$, $(s_2, 1, s_2, 1, R)$, and $(s_2, *, s_3, 1, R)$. ◀

Unfortunately, constructing Turing machines to compute relatively simple functions can be extremely demanding. For example, one Turing machine for multiplying two nonnegative integers found in many books has 31 five-tuples and 11 states. If it is challenging to construct Turing machines to compute even relatively simple functions, what hope do we have of building Turing machines for more complicated functions? One way to simplify this problem is to use a multitape Turing machine that uses more than one tape simultaneously and to build up multitape Turing machines for the composition of functions. It can be shown that for any multitape Turing machine there is a one-tape Turing machine that can do the same thing.

13.5.5 Different Types of Turing Machines

There are many variations on the definition of a Turing machine. We can expand the capabilities of a Turing machine in a wide variety of ways. For example, we can allow a Turing machine to move right, left, or not at all at each step. We can allow a Turing machine to operate on multiple tapes, using $(2 + 3n)$ -tuples to describe the Turing machine when n tapes are used. We can allow the tape to be two-dimensional, where at each step we move up, down, right, or left, not just right or left as we do on a one-dimensional tape. We can allow multiple tape heads that read different cells simultaneously. Furthermore, we can allow a Turing machine to be **nondeterministic**, by

allowing a (state, tape symbol) pair to possibly appear as the first elements in more than one five-tuple of the Turing machine. So, unlike a deterministic Turing machine, which has at most one prescribed action for a given (state, tape symbol) pair, a nondeterministic Turing machine has a set of prescribed actions for such a pair. Hence, a nondeterministic Turing machine branches into multiple copies as it runs, each of which follows a sequence of possible transitions. We can also reduce the capabilities of a Turing machine in different ways. For example, we can restrict the tape to be infinite in only one direction or we can restrict the tape alphabet to have only two symbols. All these variations of Turing machines have been studied in detail.

The crucial point is that no matter which of these variations we use, or even which combination of variations we use, we never increase or decrease the power of the machine. Anything that one of these variations can do can be done by the Turing machine defined in this section, and vice versa. The reason that these variations are useful is that sometimes they make doing some particular job much easier than if the Turing machine defined in Definition 1 were used. They never extend the capability of the machine. Sometimes it is useful to have a wide variety of Turing machines with which to work. For example, one way to show that for every nondeterministic Turing machine, there is a deterministic Turing machine that recognizes the same language is to use a deterministic Turing machine with three tapes. (For details on variations of Turing machines and demonstrations of their equivalence, see [HoMoUI01].)

Besides introducing the notion of a Turing machine, Turing also showed that it is possible to construct a single Turing machine that can simulate the computations of every Turing machine when given an encoding of this target Turing machine and its input. Such a machine is called a **universal Turing machine**. (See a book on the theory of computation, such as [Si06], for more about universal Turing machines.)

13.5.6 The Church–Turing Thesis

Links

Turing machines are relatively simple. They can have only finitely many states and they can read and write only one symbol at a time on a one-dimensional tape. But it turns out that Turing machines are extremely powerful. We have seen that Turing machines can be built to add numbers and to multiply numbers. Although it may be difficult to actually construct a Turing machine to compute a particular function that can be computed with an algorithm, such a Turing machine can always be found. This was the original goal of Turing when he invented his machines. Furthermore, there is a tremendous amount of evidence for the **Church–Turing thesis**, which states that given any problem that can be solved with an effective algorithm, there is a Turing machine that can solve this problem. The reason this is called a *thesis* rather than a theorem is that the concept of solvability by an effective algorithm is informal and imprecise, as opposed to the notion of solvability by a Turing machine, which is formal and precise. Certainly, though, any problem that can be solved using a computer with a program written in any language, perhaps using an unlimited amount of memory, should be considered effectively solvable. (Note that Turing machines have unlimited memory, unlike computers in the real world, which have only a finite amount of memory.)

Many different formal theories have been developed to capture the notion of effective computability. These include Turing’s theory and Church’s lambda-calculus, as well as theories proposed by Stephen Kleene and by E. L. Post. These theories seem quite different on the surface. The surprising thing is that they can be shown to be equivalent by demonstrating that they define exactly the same class of functions. With this evidence, it seems that Turing’s original ideas, formulated before the invention of modern computers, describe the ultimate capabilities of these machines. The interested reader should consult books on the theory of computation, such as [HoMoUI01] and [Si96], for a discussion of these different theories and their equivalence.

For the remainder of this section we will briefly explore some of the consequences of the Church–Turing thesis and we will describe the importance of Turing machines in the study of the complexity of algorithms. Our goal will be to introduce some important ideas from theoretical computer science to entice the interested student to further study. We will cover a lot of ground

quickly without providing explicit details. Our discussion will also tie together some of the concepts discussed in previous parts of the book with the theory of computation.

13.5.7 Computational Complexity, Computability, and Decidability

Throughout this book we have discussed the computational complexity of a wide variety of problems. We described the complexity of these problems in terms of the number of operations used by the most efficient algorithms that solve them. The basic operations used by algorithms differ considerably; we have measured the complexity of different algorithms in terms of bit operations, comparisons of integers, arithmetic operations, and so on. In Section 3.3, we defined various classes of problems in terms of their computational complexity. However, these definitions were not precise, because the types of operations used to measure their complexity vary so drastically. Turing machines provide a way to make the concept of computational complexity precise. If the Church–Turing thesis is true, it would then follow that if a problem can be solved using an effective algorithm, then there is a Turing machine that can solve this problem. When a Turing machine is used to solve a problem, the input to the problem is encoded as a string of symbols that is written on the tape of this Turing machine. How we encode input depends on the domain of this input. For example, as we have seen, we can encode a positive integer using a string of 1s. We can also devise ways to express pairs of integers, negative integers, and so on. Similarly, for graph algorithms, we need a way to encode graphs as strings of symbols. This can be done in many ways and can be based on adjacency lists or adjacency matrices. (We omit the details of how this is done.) However, the way input is encoded does not matter as long as it is relatively efficient, as a Turing machine can always change one encoding into another encoding. We will now use this model to make precise some of the notions concerning computational complexity that were informally introduced in Section 3.3.

The kind of problems that are most easily studied by using Turing machines are those problems that can be answered either by a “yes” or by a “no.”

Definition 3

A decision problem asks whether statements from a particular class of statements are true. Decision problems are also known as yes-or-no problems.

Given a decision problem, we would like to know whether there is an algorithm that can determine whether statements from the class of statements it addresses are true. For example, consider the class of statements each of which asks whether a particular integer n is prime. This is a decision problem because the answer to the question “Is n prime?” is either yes or no. Given this decision problem, we can ask whether there is an algorithm that can decide whether each of the statements in the decision problem is true, that is, given an integer n , deciding whether n is prime. The answer is that there is such an algorithm. In particular, in Section 3.5 we discussed the algorithm that determines whether a positive integer n is prime by checking whether it is divisible by primes not exceeding its square root. (There are many other algorithms for determining whether a positive integer is prime.)

The set of inputs for which the answer to the yes–no problem is “yes” is a subset of the set of possible inputs, that is, it is a subset of the set of strings of the input alphabet. In other words, solving a yes–no problem is the same as recognizing the language consisting of all bit strings that represent input values to the problem leading to the answer “yes.” Consequently, solving a yes–no problem is the same as recognizing the language corresponding to the input values for which the answer to the problem is “yes.”

DECIDABILITY When there is an effective algorithm that decides whether instances of a decision problem are true, we say that this problem is **solvable** or **decidable**. For instance, the problem of determining whether a positive integer is prime is a solvable problem. However, if no

effective algorithm exists for solving a problem, then we say the problem is **unsolvable** or **undecidable**. To show that a decision problem is solvable we need only construct an algorithm that can determine whether statements of the particular class are true. On the other hand, to show that a decision problem is unsolvable we need to prove that no such algorithm exists. (The fact that we tried to find such an algorithm but failed, does not prove the problem is unsolvable.)

By studying only decision problems, it may seem that we are studying only a small set of problems of interest. However, most problems can be recast as decision problems. Recasting the types of problems we have studied in this book as decision problems can be quite complicated, so we will not go into the details of this process here. The interested reader can consult references on the theory of computation, such as [Wo87], which, for example, explains how to recast the traveling salesperson problem (described in Section 9.6) as a decision problem. (To recast the traveling salesman problem as a decision problem, we first consider the decision problem that asks whether there is a Hamilton circuit of weight not exceeding k , where k is a positive integer. With some additional effort it is possible to use answers to this question for different values of k to find the smallest possible weight of a Hamilton circuit.)

In Section 3.1 we introduced the halting problem and proved that it is an unsolvable problem. That discussion was somewhat informal because the notion of a procedure was not precisely defined. A precise definition of the halting problem can be made in terms of Turing machines.

Definition 4

The *halting problem* is the decision problem that asks whether a Turing machine T eventually halts when given an input string x .

With this definition of the halting problem, we have Theorem 1.

THEOREM 1

The halting problem is an unsolvable decision problem. That is, no Turing machine exists that, when given an encoding of a Turing machine T and its input string x as input, can determine whether T eventually halts when started with x written on its tape.

The proof of Theorem 1 given in Section 3.1 for the informal definition of the halting problem still applies here.

Other examples of unsolvable problems include:

- (i) the problem of determining whether two context-free grammars generate the same set of strings;
- (ii) the problem of determining whether a given set of tiles can be used with repetition allowed to cover the entire plane without overlap; and
- (iii) *Hilbert's Tenth Problem*, which asks whether there are integer solutions to a given polynomial equation with integer coefficients. (This question occurs tenth on the influential list of 23 unsolved problems Hilbert posed in 1900. Hilbert envisioned that the work done to solve these problems would help further the progress of mathematics in the twentieth century. The unsolvability of Hilbert's Tenth Problem was established in 1970 by Yuri Matiyasevich.)



COMPUTABILITY A function that can be computed by a Turing machine is called **computable** and a function that cannot be computed by a Turing machine is called **uncomputable**. It is fairly straightforward, using a countability argument, to show that there are number-theoretic functions that are not computable (see Exercise 39 in Section 2.5). However, it is not so easy to actually produce such a function. The **busy beaver function** defined in the preamble to Exercise 31 is an example of an uncomputable function. One way to show that the busy beaver function is not computable is to show that it grows faster than any computable function. (See Exercise 32.)

Note that every decision problem can be reformulated as the problem of computing a function, namely, the function that has the value 1 when the answer to the problem is “yes” and that has the value 0 when the answer to the problem is “no.” A decision problem is solvable if and only if the corresponding function constructed in this way is computable.

THE CLASSES P AND NP In Section 3.3 we informally defined the classes of problems called P and NP. We are now able to define these concepts precisely using the notions of deterministic and nondeterministic Turing machines.

We first elaborate on the difference between a deterministic Turing machine and a nondeterministic Turing machine. The Turing machines we have studied in this section have all been deterministic. In a deterministic Turing machine $T = (S, I, f, s_0)$, transition rules are defined by the partial function f from $S \times I$ to $S \times I \times \{R, L\}$. Consequently, when transition rules of the machine are represented as five-tuples of the form (s, x, s', x', d) , where s is the current state, x is the current tape symbol, s' is the next state, x' is the symbol that replaces x on the tape, and d is the direction the machine moves on the tape, no two transition rules begin with the same pair (s, x) .

In a nondeterministic Turing machine, allowed steps are defined using a relation consisting of five-tuples rather than using a partial function. The restriction that no two transition rules begin with the same pair (s, x) is eliminated; that is, there may be more than one transition rule beginning with each (state, tape symbol) pair. Consequently, in a nondeterministic Turing machine, there is a choice of transitions for some pairs of the current state and the tape symbol being read. At each step of the operation of a nondeterministic Turing machine, the machine picks one of the different choices of the transition rules that begin with the current state and tape symbol pair. This choice can be considered to be a “guess” of which step to use. Just as for deterministic Turing machines, a nondeterministic Turing machine halts when there is no transition rule in its definition that begins with the current state and tape symbol. Given a nondeterministic Turing machine T , we say that a string x is recognized by T if and only if there exists some sequence of transitions of T that ends in a final state when the machine starts in the initial position with x written on the tape. The nondeterministic Turing machine T recognizes the set A if x is recognized by T if and only if $x \in A$. The nondeterministic Turing machine T is said to solve a decision problem if it recognizes the set consisting of all input values for which the answer to the decision problem is yes.

Definition 5

A decision problem is in P, the *class of polynomial-time problems*, if it can be solved by a deterministic Turing machine in polynomial time in terms of the size of its input. That is, a decision problem is in P if there is a deterministic Turing machine T that solves the decision problem and a polynomial $p(n)$ such that for all integers n , T halts in a final state after no more than $p(n)$ transitions whenever the input to T is a string of length n . A decision problem is in NP, the *class of nondeterministic polynomial-time problems*, if it can be solved by a nondeterministic Turing machine in polynomial time in terms of the size of its input. That is, a decision problem is in NP if there is a nondeterministic Turing machine T that solves the problem and a polynomial $p(n)$ such that for all integers n , T halts for every choice of transitions after no more than $p(n)$ transitions whenever the input to T is a string of length n .

Links



©University Archives.
Department of Rare Books
and Special Collections.
Princeton University Library

ALONZO CHURCH (1903–1995) Alonzo Church was born in Washington, D.C. He studied at Göttingen under Hilbert and later in Amsterdam. He was a member of the faculty at Princeton University from 1927 until 1967, when he moved to UCLA. Church was one of the founding members of the Association for Symbolic Logic. He made many substantial contributions to the theory of computability, including his solution to the decision problem, his invention of the lambda-calculus, and his statement of what is now known as the Church–Turing thesis. Among Church’s students were Stephen Kleene and Alan Turing. He published articles past his 90th birthday.

Problems in P are called **tractable**, whereas problems not in P are called **intractable**. For a problem to be in P , a deterministic Turing machine must exist that can decide in polynomial time whether a particular statement of the class addressed by the decision problem is true. For example, determining whether an item is in a list of n elements is a tractable problem. (We will not provide details on how this fact can be shown; the basic ideas used in the analyses of algorithms earlier in the text can be adapted when Turing machines are employed.) For a problem to be in NP , it is necessary only that there be a nondeterministic Turing machine that, when given a true statement from the set of statements addressed by the problem, can verify its truth in polynomial time by making the correct guess at each step from the set of allowable steps corresponding to the current state and tape symbol. The problem of determining whether a given graph has a Hamilton circuit is an NP problem, because a nondeterministic Turing machine can easily verify that a simple circuit in a graph passes through each vertex exactly once. It can do this by making a series of correct guesses corresponding to successively adding edges to form the circuit. Because every deterministic Turing machine can also be considered to be a nondeterministic Turing machine where each (state, tape symbol) pair occurs in exactly one transition rule defining the machine, every problem in P is also in NP . In symbols, $P \subseteq NP$.

One of the most perplexing open questions in theoretical computer science is whether every problem in NP is also in P , that is, whether $P = NP$. As mentioned in Section 3.3, there is an important class of problems, the class of NP -complete problems, such that a problem is in this class if it is in the class NP and if it can be shown that if it is also in the class P , then *every* problem in the class NP must also be in the class P . That is, a problem is NP -complete if the existence of a polynomial-time algorithm for solving it implies the existence of a polynomial-time algorithm for every problem in NP . In this book we have discussed several different NP -complete problems, such as determining whether a simple graph has a Hamilton circuit and determining whether a proposition in n -variables is a tautology.

Exercises

1. Let T be the Turing machine defined by the five-tuples: $(s_0, 0, s_1, 1, R)$, $(s_0, 1, s_1, 0, R)$, $(s_0, B, s_1, 0, R)$, $(s_1, 0, s_2, 1, L)$, $(s_1, 1, s_1, 0, R)$, and $(s_1, B, s_2, 0, L)$. For each of these initial tapes, determine the final tape when T halts, assuming that T begins in initial position.

a)	...	B	B	0	0	1	1	B	B	...
b)	...	B	B	1	0	1	B	B	B	...
c)	...	B	B	1	1	B	0	1	B	...
d)	...	B	B	B	B	B	B	B	B	...

2. Let T be the Turing machine defined by the five-tuples: $(s_0, 0, s_1, 0, R)$, $(s_0, 1, s_1, 0, L)$, $(s_0, B, s_1, 1, R)$, $(s_1, 0, s_2, 1, R)$, $(s_1, 1, s_1, 1, R)$, $(s_1, B, s_2, 0, R)$, and $(s_2, B, s_3, 0, R)$. For each of these initial tapes, determine the final tape when T halts, assuming that T begins in initial position.

a)	...	B	B	0	1	0	1	B	B	...
b)	...	B	B	1	1	1	B	B	B	...

c)	...	B	B	0	0	B	0	0	B	...
d)	...	B	B	B	B	B	B	B	B	...

3. What does the Turing machine described by the five-tuples $(s_0, 0, s_0, 0, R)$, $(s_0, 1, s_1, 0, R)$, (s_0, B, s_2, B, R) , $(s_1, 0, s_1, 0, R)$, $(s_1, 1, s_0, 1, R)$, and (s_1, B, s_2, B, R) do when given
- 11 as input?
 - an arbitrary bit string as input?
4. What does the Turing machine described by the five-tuples $(s_0, 0, s_0, 1, R)$, $(s_0, 1, s_0, 1, R)$, (s_0, B, s_1, B, L) , $(s_1, 1, s_2, 1, R)$, do when given
- 101 as input?
 - an arbitrary bit string as input?
5. What does the Turing machine described by the five-tuples $(s_0, 1, s_1, 0, R)$, $(s_1, 1, s_1, 1, R)$, $(s_1, 0, s_2, 0, R)$, $(s_2, 0, s_3, 1, L)$, $(s_2, 1, s_2, 1, R)$, $(s_3, 1, s_3, 1, L)$, $(s_3, 0, s_4, 0, L)$, $(s_4, 1, s_4, 1, L)$, and $(s_4, 0, s_0, 1, R)$ do when given
- 11 as input?
 - a bit string consisting entirely of 1s as input?
6. Construct a Turing machine with tape symbols 0, 1, and B that, when given a bit string as input, adds a 1 to

- the end of the bit string and does not change any of the other symbols on the tape.
7. Construct a Turing machine with tape symbols 0, 1, and B that, given a bit string as input, replaces the first 0 with a 1 and does not change any of the other symbols on the tape.
 8. Construct a Turing machine with tape symbols 0, 1, and B that, given a bit string as input, replaces all 0s on the tape with 1s and does not change any of the 1s on the tape.
 9. Construct a Turing machine with tape symbols 0, 1, and B that, given a bit string as input, replaces all but the leftmost 1 on the tape with 0s and does not change any of the other symbols on the tape.
 10. Construct a Turing machine with tape symbols 0, 1, and B that, given a bit string as input, replaces the first two consecutive 1s on the tape with 0s and does not change any of the other symbols on the tape.
 11. Construct a Turing machine that recognizes the set of all bit strings that end with a 0.
 12. Construct a Turing machine that recognizes the set of all bit strings that contain at least two 1s.
 13. Construct a Turing machine that recognizes the set of all bit strings that contain an even number of 1s.
 14. Show at each step the contents of the tape of the Turing machine in Example 3 starting with each of these strings.
a) 0011 b) 00011 c) 101100 d) 000111
 15. Explain why the Turing machine in Example 3 recognizes a bit string if and only if this string is of the form $0^n 1^n$ for some positive integer n .
 - *16. Construct a Turing machine that recognizes the set $\{0^{2^n} 1^n \mid n \geq 0\}$.
 - *17. Construct a Turing machine that recognizes the set $\{0^n 1^n 2^n \mid n \geq 0\}$.
 18. Construct a Turing machine that computes the function $f(n) = n + 2$ for all nonnegative integers n .
 19. Construct a Turing machine that computes the function $f(n) = n - 3$ if $n \geq 3$ and $f(n) = 0$ for $n = 0, 1, 2$ for all nonnegative integers n .
 20. Construct a Turing machine that computes the function $f(n) = n \bmod 3$ for every nonnegative integer n .
 21. Construct a Turing machine that computes the function $f(n) = 3$ if $n \geq 5$ and $f(n) = 0$ if $n = 0, 1, 2, 3$, or 4.
 22. Construct a Turing machine that computes the function $f(n) = 2n$ for all nonnegative integers n .
 23. Construct a Turing machine that computes the function $f(n) = 3n$ for all nonnegative integers n .
 24. Construct a Turing machine that computes the function $f(n_1, n_2) = n_2 + 2$ for all pairs of nonnegative integers n_1 and n_2 .
 - *25. Construct a Turing machine that computes the function $f(n_1, n_2) = \min(n_1, n_2)$ for all nonnegative integers n_1 and n_2 .
 26. Construct a Turing machine that computes the function $f(n_1, n_2) = n_1 + n_2 + 1$ for all nonnegative integers n_1 and n_2 .
- Suppose that T_1 and T_2 are Turing machines with disjoint sets of states S_1 and S_2 and with transition functions f_1 and f_2 , respectively. We can define the Turing machine $T_1 T_2$, the **composite** of T_1 and T_2 , as follows. The set of states of $T_1 T_2$ is $S_1 \cup S_2$. $T_1 T_2$ begins in the start state of T_1 . It first executes the transitions of T_1 using f_1 up to, but not including, the step at which T_1 would halt. Then, for all moves for which T_1 halts, it executes the same transitions of T_1 except that it moves to the start state of T_2 . From this point on, the moves of $T_1 T_2$ are the same as the moves of T_2 .
27. By finding the composite of the Turing machines you constructed in Exercises 18 and 22, construct a Turing machine that computes the function $f(n) = 2n + 2$.
 28. By finding the composite of the Turing machines you constructed in Exercises 18 and 23, construct a Turing machine that computes the function $f(n) = 3(n + 2) = 3n + 6$.
 29. Which of the following problems is a decision problem?
 - a) What is the smallest prime greater than n ?
 - b) Is a graph G bipartite?
 - c) Given a set of strings, is there a finite-state automaton that recognizes this set of strings?
 - d) Given a checkerboard and a particular type of polyomino (see Section 1.8), can this checkerboard be tiled using polyominoes of this type?
 30. Which of the following problems is a decision problem?
 - a) Is the sequence $a_1, a_2, \dots, a_n$ of positive integers in increasing order?
 - b) Can the vertices of a simple graph G be colored using three colors so that no two adjacent vertices are the same color?
 - c) What is the vertex of highest degree in a graph G ?
 - d) Given two finite-state machines, do these machines recognize the same language?
- Let $B(n)$ be the maximum number of 1s that a Turing machine with n states with the alphabet $\{1, B\}$ may print on a tape that is initially blank. The problem of determining $B(n)$ for particular values of n is known as the **busy beaver problem**. This problem was first studied by Tibor Rado in 1962. Currently it is known that $B(2) = 4$, $B(3) = 6$, and $B(4) = 13$, but $B(n)$ is not known for $n \geq 5$. $B(n)$ grows rapidly; it is known that $B(5) \geq 4098$ and $B(6) \geq 3.5 \times 10^{18267}$.
- *31. Show that $B(2)$ is at least 4 by finding a Turing machine with two states and alphabet $\{1, B\}$ that halts with four consecutive 1s on the tape.
 - **32. Show that the function $B(n)$ cannot be computed by any Turing machine. [Hint: Assume that there is a Turing machine that computes $B(n)$ in binary. Build a Turing machine T that, starting with a blank tape, writes n down in binary, computes $B(n)$ in binary, and converts $B(n)$ from binary to unary. Show that for sufficiently large n , the number of states of T is less than $B(n)$, leading to a contradiction.]

Key Terms and Results

TERMS

alphabet (or vocabulary): a set that contains elements used to form strings

language: a subset of the set of all strings over an alphabet

phrase-structure grammar (V, T, S, P) : a description of a language containing an alphabet V , a set of terminal symbols T , a start symbol S , and a set of productions P

the production $w \rightarrow w_1$: w can be replaced by w_1 whenever it occurs in a string in the language

$w_1 \Rightarrow w_2$ (w_2 is directly derivable from w_1): w_2 can be obtained from w_1 using a production to replace a string in w_1 with another string

$w_1 \stackrel{*}{\Rightarrow} w_2$ (w_2 is derivable from w_1): w_2 can be obtained from w_1 using a sequence of productions to replace strings by other strings

type 0 grammar: any phrase-structure grammar

type 1 grammar: a phrase-structure grammar in which every production is of the form $w_1 \rightarrow w_2$, where $w_1 = lAr$ and $w_2 = lwr$, where $A \in N$, $l, r, w \in (N \cup T)^*$ and $w \neq \lambda$, or $w_1 = S$ and $w_2 = \lambda$ as long as S is not on the right-hand side of another production

type 2, or context-free, grammar: a phrase-structure grammar in which every production is of the form $A \rightarrow w_1$, where A is a nonterminal symbol

type 3, or regular, grammar: a phrase-structure grammar where every production is of the form $A \rightarrow aB$, $A \rightarrow a$, or $S \rightarrow \lambda$, where A and B are nonterminal symbols, S is the start symbol, and a is a terminal symbol

derivation (or parse) tree: an ordered rooted tree where the root represents the starting symbol of a type 2 grammar, internal vertices represent nonterminals, leaves represent terminals, and the children of a vertex are the symbols on the right side of a production, in order from left to right, where the symbol represented by the parent is on the left-hand side

Backus–Naur form: a description of a context-free grammar in which all productions having the same nonterminal as their left-hand side are combined with the different right-hand sides of these productions, each separated by a bar, with nonterminal symbols enclosed in angular brackets and the symbol $\rightarrow$ replaced by $::=$

finite-state machine (S, I, O, f, g, s_0) (or a Mealy machine): a six-tuple containing a set S of states, an input alphabet I , an output alphabet O , a transition function f that assigns a next state to every pair of a state and an input, an output function g that assigns an output to every pair of a state and an input, and a starting state s_0

AB (concatenation of A and B): the set of all strings formed by concatenating a string in A and a string in B in that order

A^* (Kleene closure of A): the set of all strings made up by concatenating arbitrarily many strings from A

deterministic finite-state automaton (S, I, f, s_0, F) : a five-tuple containing a set S of states, an input alphabet I , a transition function f that assigns a next state to every pair of a state and an input, a starting state s_0 , and a set of final states F

nondeterministic finite-state automaton (S, I, f, s_0, F) : a five-tuple containing a set S of states, an input alphabet I , a transition function f that assigns a set of possible next states to every pair of a state and an input, a starting state s_0 , and a set of final states F

language recognized by an automaton: the set of input strings that take the start state to a final state of the automaton

regular expression: an expression defined recursively by specifying that $\emptyset$, λ , and x , for all x in the input alphabet, are regular expressions, and that (AB) , $(A \cup B)$, and A^* are regular expressions when A and B are regular expressions

regular set: a set defined by a regular expression (see page 820)

Turing machine $T = (S, I, f, s_0)$: a four-tuple consisting of a finite set S of states, an alphabet I containing the blank symbol B , a partial function f from $S \times I$ to $S \times I \times \{R, L\}$, and a starting state s_0

nondeterministic Turing machine: a Turing machine that may have more than one transition rule corresponding to each (state, tape symbol) pair

decision problem: a problem that asks whether statements from a particular class of statements are true

solvable problem: a problem with the property that there is an effective algorithm that can solve all instances of the problem

unsolvable problem: a problem with the property that no effective algorithm exists that can solve all instances of the problem

computable function: a function whose values can be computed using a Turing machine

uncomputable function: a function whose values cannot be computed using a Turing machine

P, the class of polynomial-time problems: the class of problems that can be solved by a deterministic Turing machine in polynomial time in terms of the size of the input

NP, the class of nondeterministic polynomial-time problems: the class of problems that can be solved by a nondeterministic Turing machine in polynomial time in terms of the size of the input

NP-complete: a subset of the class of NP problems with the property that if any one of them is in the class P, then all problems in NP are in the class P

RESULTS

For every nondeterministic finite-state automaton there is a deterministic finite-state automaton that recognizes the same set.

Review Questions

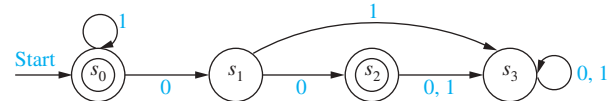
- Define a phrase-structure grammar.
 - What does it mean for a string to be derivable from a string w by a phrase-structure grammar G ?
- What is the language generated by a phrase-structure grammar G ?
 - What is the language generated by the grammar G with vocabulary $\{S, 0, 1\}$, set of terminals $T = \{0, 1\}$, starting symbol S , and productions $S \rightarrow 000S, S \rightarrow 1$?
 - Give a phrase-structure grammar that generates the set $\{01^n \mid n = 0, 1, 2, \dots\}$.
- Define a type 1 grammar.
 - Give an example of a grammar that is not a type 1 grammar.
 - Define a type 2 grammar.
 - Give an example of a grammar that is not a type 2 grammar but is a type 1 grammar.
 - Define a type 3 grammar.
 - Give an example of a grammar that is not a type 3 grammar but is a type 2 grammar.
- Define a regular grammar.
 - Define a regular language.
 - Show that the set $\{0^m 1^n \mid m, n = 0, 1, 2, \dots\}$ is a regular language.
- What is Backus–Naur form?
 - Give an example of the Backus–Naur form of the grammar for a subset of English of your choice.
- What is a finite-state machine?
 - Show how a vending machine that accepts only quarters and dispenses a soft drink after 75 cents has been deposited can be modeled using a finite-state machine.

Kleene's theorem: A set is regular if and only if there is a finite-state automaton that recognizes it.

A set is regular if and only if it is generated by a regular grammar.

The halting problem is unsolvable.

- Find the set of strings recognized by the deterministic finite-state automaton shown here.



- Construct a deterministic finite-state automaton that recognizes the set of bit strings that start with 1 and end with 1.
- What is the Kleene closure of a set of strings?
 - Find the Kleene closure of the set $\{11, 0\}$.
- Define a finite-state automaton.
 - What does it mean for a string to be recognized by a finite-state automaton?
- Define a nondeterministic finite-state automaton.
 - Show that given a nondeterministic finite-state automaton, there is a deterministic finite-state automaton that recognizes the same language.
- Define the set of regular expressions over a set I .
 - Explain how regular expressions are used to represent regular sets.
- State Kleene's theorem.
- Show that a set is generated by a regular grammar if and only if it is a regular set.
- Give an example of a set not recognized by a finite-state automaton. Show that no finite-state automaton recognizes it.
- Define a Turing machine.
- Describe how Turing machines are used to recognize sets.
- Describe how Turing machines are used to compute number-theoretic functions.
- What is an unsolvable decision problem? Give an example of such a problem.

Supplementary Exercises

- Find a phrase-structure grammar that generates each of these languages.
 - the set of bit strings of the form $0^{2n}1^{3n}$, where n is a nonnegative integer
 - the set of bit strings with twice as many 0s as 1s
 - the set of bit strings of the form w^2 , where w is a bit string
- Find a phrase-structure grammar that generates the set $\{0^{2^n} \mid n \geq 0\}$.

For Exercises 3 and 4, let $G = (V, T, S, P)$ be the context-free grammar with $V = \{(,), S, A, B\}$, $T = \{(,)\}$, starting symbol S , and productions $S \rightarrow A, A \rightarrow AB, A \rightarrow B, B \rightarrow (A)$, and $B \rightarrow (), S \rightarrow \lambda$.

- Construct the derivation trees of these strings.
 - $(())$
 - $((()))$
 - $(((())))$
- Show that $L(G)$ is the set of all balanced strings of parentheses, defined in the preamble to Supplementary Exercise 59 in Chapter 5.

A context-free grammar is **ambiguous** if there is a word in $L(G)$ with two derivations that produce different derivation trees, considered as ordered, rooted trees.

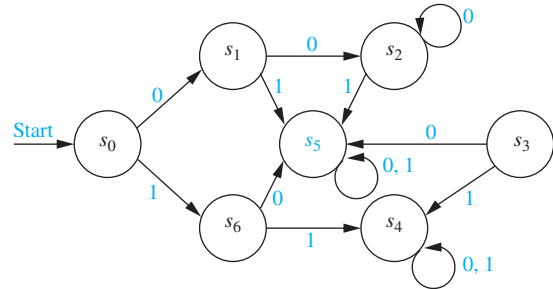
5. Show that the grammar $G = (V, T, S, P)$ with $V = \{0, S\}$, $T = \{0\}$, starting state S , and productions $S \rightarrow 0S$, $S \rightarrow S0$, and $S \rightarrow 0$ is ambiguous by constructing two different derivation trees for 0^3 .
6. Show that the grammar $G = (V, T, S, P)$ with $V = \{0, S\}$, $T = \{0\}$, starting state S , and productions $S \rightarrow 0S$ and $S \rightarrow 0$ is unambiguous.
7. Suppose that A and B are finite subsets of V^* , where V is an alphabet. Is it necessarily true that $|AB| = |BA|$?
8. Prove or disprove each of these statements for subsets A , B , and C of V^* , where V is an alphabet.
 - a) $A(B \cup C) = AB \cup AC$
 - b) $A(B \cap C) = AB \cap AC$
 - c) $(AB)C = A(BC)$
 - d) $(A \cup B)^* = A^* \cup B^*$
9. Suppose that A and B are subsets of V^* , where V is an alphabet. Does it follow that $A \subseteq B$ if $A^* \subseteq B^*$?
10. What set of strings with symbols in the set $\{0, 1, 2\}$ is represented by the regular expression $(2^*)(0 \cup (12^*))^*$?

The **star height** $h(E)$ of a regular expression over the set I is defined recursively by

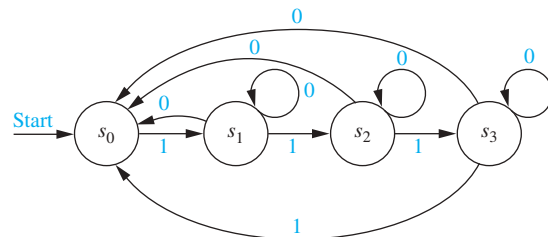
$$\begin{aligned} h(\emptyset) &= 0; \\ h(x) &= 0 \text{ if } x \in I; \\ h(E_1 \cup E_2) &= h(E_1 E_2) = \max(h(E_1), h(E_2)) \\ &\quad \text{if } E_1 \text{ and } E_2 \text{ are regular expressions;} \\ h(E^*) &= h(E) + 1 \text{ if } E \text{ is a regular expression.} \end{aligned}$$

11. Find the star height of each of these regular expressions.
 - a) 0^*1
 - b) 0^*1^*
 - c) $(0^*01)^*$
 - d) $((0^*1)^*)^*$
 - e) $(010^*)(1^*01^*)((01)^*(10)^*)^*$
 - f) $(((((0^*)1^*)0^*)1^*))^*)^*$
- *12. For each of these regular expressions find a regular expression that represents the same language with minimum star height.
 - a) $(0^*1^*)^*$
 - b) $(0(01^*0)^*)^*$
 - c) $(0^* \cup (01)^* \cup 1^*)^*$
13. Construct a finite-state machine with output that produces an output of 1 if the bit string read so far as input contains four or more 1s. Then construct a deterministic finite-state automaton that recognizes this set.
14. Construct a finite-state machine with output that produces an output of 1 if the bit string read so far as input contains four or more consecutive 1s. Then construct a deterministic finite-state automaton that recognizes this set.
15. Construct a finite-state machine with output that produces an output of 1 if the bit string read so far as input ends with four or more consecutive 1s. Then construct a deterministic finite-state automaton that recognizes this set.

16. A state s' in a finite-state machine is said to be **reachable** from state s if there is an input string x such that $f(s, x) = s'$. A state s is called **transient** if there is no nonempty input string x with $f(s, x) = s$. A state s is called a **sink** if $f(s, x) = s$ for all input strings x . Answer these questions about the finite-state machine with the state diagram illustrated here.



- a) Which states are reachable from s_0 ?
- b) Which states are reachable from s_2 ?
- c) Which states are transient?
- d) Which states are sinks?
- *17. Suppose that S , I , and O are finite sets such that $|S| = n$, $|I| = k$, and $|O| = m$.
 - a) How many different finite-state machines (Mealy machines) $M = (S, I, O, f, g, s_0)$ can be constructed, where the starting state s_0 can be arbitrarily chosen?
 - b) How many different Moore machines $M = (S, I, O, f, g, s_0)$ can be constructed, where the starting state s_0 can be arbitrarily chosen?
- *18. Suppose that S and I are finite sets such that $|S| = n$ and $|I| = k$. How many different finite-state automata $M = (S, I, f, s_0, F)$ are there where the starting state s_0 and the subset F of S consisting of final states can be chosen arbitrarily?
 - a) if the automata are deterministic?
 - b) if the automata may be nondeterministic? (Note: This includes deterministic automata.)
19. Construct a deterministic finite-state automaton that is equivalent to the nondeterministic automaton with the state diagram shown here.



20. What is the language recognized by the automaton in Exercise 19?
21. Construct finite-state automata that recognize these sets.
 - a) $0^*(10)^*$
 - b) $(01 \cup 111)^*10^*(0 \cup 1)$
 - c) $(001 \cup (11)^*)^*$

- *22. Find regular expressions that represent the set of all strings of 0s and 1s
 - a) made up of blocks of even numbers of 1s interspersed with odd numbers of 0s.
 - b) with at least two consecutive 0s or three consecutive 1s.
 - c) with no three consecutive 0s or two consecutive 1s.
- *23. Show that if A is a regular set, then so is $\bar{A}$.
- *24. Show that if A and B are regular sets, then so is $A \cap B$.
- *25. Find finite-state automata that recognize these sets of strings of 0s and 1s.
 - a) the set of all strings that start with no more than three consecutive 0s and contain at least two consecutive 1s
 - b) the set of all strings with an even number of symbols that do not contain the pattern 101
 - c) the set of all strings with at least three blocks of two or more 1s and at least two 0s
- *26. Show that $\{0^{2^n} \mid n \in \mathbb{N}\}$ is not regular. You may use the pumping lemma given in Exercise 22 of Section 13.4.
- *27. Show that $\{1^p \mid p \text{ is prime}\}$ is not regular. You may use the pumping lemma given in Exercise 22 of Section 13.4.
- *28. There is a result for context-free languages analogous to the pumping lemma for regular sets. Suppose that $L(G)$ is the language recognized by a context-free language G . This result states that there is a constant N such that if z is a word in $L(G)$ with $l(z) \geq N$, then z can be written as $uvwxy$, where $l(vwx) \leq N$, $l(vx) \geq 1$, and uv^iwx^iy belongs to $L(G)$ for $i = 0, 1, 2, 3, \dots$. Use this result to show that there is no context-free grammar G with $L(G) = \{0^n 1^n 2^n \mid n = 0, 1, 2, \dots\}$.
- *29. Construct a Turing machine that computes the function $f(n_1, n_2) = \max(n_1, n_2)$.
- *30. Construct a Turing machine that computes the function $f(n_1, n_2) = n_2 - n_1$ if $n_2 \geq n_1$ and $f(n_1, n_2) = 0$ if $n_2 < n_1$.

Computer Projects

Write programs with these input and output.

1. Given the productions in a phrase-structure grammar, determine which type of grammar this is in the Chomsky classification scheme.
2. Given the productions of a phrase-structure grammar, find all strings that are generated using twenty or fewer applications of its production rules.
3. Given the Backus–Naur form of a type 2 grammar, find all strings that are generated using twenty or fewer applications of the rules defining it.
- *4. Given the productions of a context-free grammar and a string, produce a derivation tree for this string if it is in the language generated by this grammar.
5. Given the state table of a Moore machine and an input string, produce the output string generated by the machine.
6. Given the state table of a Mealy machine and an input string, produce the output string generated by the machine.
7. Given the state table of a deterministic finite-state automaton and a string, decide whether this string is recognized by the automaton.
8. Given the state table of a nondeterministic finite-state automaton and a string, decide whether this string is recognized by the automaton.
- *9. Given the state table of a nondeterministic finite-state automaton, construct the state table of a deterministic finite-state automaton that recognizes the same language.
- **10. Given a regular expression, construct a nondeterministic finite-state automaton that recognizes the set that this expression represents.
11. Given a regular grammar, construct a finite-state automaton that recognizes the language generated by this grammar.
12. Given a finite-state automaton, construct a regular grammar that generates the language recognized by this automaton.
- *13. Given a Turing machine, find the output string produced by a given input string.

Computations and Explorations

Use a computational program or programs you have written to do these exercises.

1. Solve the busy beaver problem for two states by testing all possible Turing machines with two states and alphabet $\{1, B\}$.
- *2. Solve the busy beaver problem for three states by testing all possible Turing machines with three states and alphabet $\{1, B\}$.
- **3. Find a busy beaver machine with four states by testing all possible Turing machines with four states and alphabet $\{1, B\}$.
- **4. Make as much progress as you can toward finding a busy beaver machine with five states.
- **5. Make as much progress as you can toward finding a busy beaver machine with six states.

Writing Projects

Respond to these with essays using outside sources.

1. Describe how the growth of certain types of plants can be modeled using a Lindenmeyer system. Such a system uses a grammar with productions modeling the different ways plants can grow.
2. Describe the Backus–Naur form (and extended Backus–Naur form) rules used to specify the syntax of a programming language, such as Java, LISP, or Ada, or the database language SQL.
3. Explain how finite-state machines are used by spell-checkers.
4. Explain how finite-state machines are used in the study of network protocols.
5. Explain how finite-state machines are used in speech recognition programs.
6. Compare the use of Moore machines versus Mealy machines in the design of hardware systems and computer software.
7. Explain the concept of minimizing finite-state automata. Give an algorithm that carries out this minimization.
8. Give the definition of cellular automata. Explain their applications. Use the Game of Life as an example.
9. Define a pushdown automaton. Explain how pushdown automata are used to recognize sets. Which sets are recognized by pushdown automata? Provide an outline of a proof justifying your answer.
10. Define a linear-bounded automaton. Explain how linear-bounded automata are used to recognize sets. Which sets are recognized by linear-bounded automata? Provide an outline of a proof justifying your answer.
11. Look up Turing’s original definition of what we now call a Turing machine. What was his motivation for defining these machines?
12. Describe the concept of the universal Turing machine. Explain how such a machine can be built.
13. Explain the kinds of applications in which nondeterministic Turing machines are used instead of deterministic Turing machines.
14. Show that a Turing machine can simulate any action of a nondeterministic Turing machine.
15. Show that a set is recognized by a Turing machine if and only if it is generated by a phrase-structure grammar.
16. Describe the basic concepts of the lambda-calculus and explain how it is used to study computability of functions.
17. Show that a Turing machine as defined in this chapter can do anything a Turing machine with n tapes can do.
18. Show that a Turing machine with a tape infinite in one direction can do anything a Turing machine with a tape infinite in both directions can do.